Experimental Dialog Correlation and Recovery Across Agent Adapters

Proposed Structure

Team Code: A3-PS021-TC047

Team Name: 404 Crew

AIORI-3 Hackathon • 6G & Future Networks

Mentor:
Bharani H,
bharani1@srmist.edu.in

Students:
Subehaa SB
us4373@srmist.edu.in
Prem J
pj6631@srmist.edu.in

AIORI Hackathon 3
October 5, 2026

Contents

1. Problem Description
2. Background
3. Solution Proposed
4. Optimizations Proposed by the Team
5. Solution Architecture and Design
6. Timeline of Delivery
7. References
8. Conclusion

Pseudocode for the Mentor Demonstration

The following short flow shows the main recovery and duplicate-request logic that will be demonstrated to the mentor:

```
RECEIVE REQUEST
IF REQUEST_ID WAS ALREADY PROCESSED:
REJECT REQUEST ELSE:
   SAVE REQUEST AND CURRENT DIALOG STATE
   PROCESS TASK
   MARK REQUEST AS COMPLETED IF
AGENT RECONNECTS:
```

LOAD SAVED DIALOG STATE
CONTINUE THE SAME TASK

1. Problem Description

When two agents are working on the same task, a restart or a dropped connection can make the next request look like a new one. That can leave the task half-finished or cause the same operation to happen twice.

Our prototype will keep the task identifiable from start to finish, even when one adapter goes down and comes back.

- Connect two mock agent adapters.
- Give each task a dialog ID and request ID.
- Keep task state after an adapter restart.
- Reject a request that has already been processed.
- Test five disconnect and retry situations.
- Allow only the state changes defined for the task.

2. Background

AI agents are increasingly being used to handle tasks that involve multiple steps and communication between different services. In many cases, a task may pass between agents built using different frameworks or run on separate services. For such systems to work reliably, they need to keep track of the task even when an agent becomes temporarily unavailable.

A problem occurs when an agent disconnects, restarts, or fails to respond while a task is in progress. When communication resumes, the system may lose track of the earlier conversation or process the same request again.

This can lead to duplicated operations, incomplete tasks, or inconsistent task states. These issues become more difficult to handle when multiple agents are involved.

The proposed project, PS-021, focuses on improving how dialog information and task identity are maintained across agent adapters. The system will use dialog IDs, task IDs, request tracking, and saved state to identify ongoing work. If an agent disconnects or restarts, the system should be able to restore the saved state and continue the same task. It will also check whether a request has already been processed to avoid unnecessary duplicate execution.

The team plans to demonstrate this approach using two mock agent adapters and a set of simulated disconnect and retry scenarios. The aim is to show how basic state management, request deduplication, and recovery mechanisms can make communication between agents more reliable.

3. Solution Proposed

We will build a small service around two mock agent adapters. Spring Boot will keep track of the dialog and task state. Two FastAPI services will act as the adapters. An LLM can be used by the agents to turn a request into a structured action, but the recovery decisions will stay in our backend.

The backend will save the dialog and request information in a database. If an adapter reconnects and sends the same request again, the backend will check the request ID before allowing the operation to run.

What the prototype will show

- Agent A sends a task to Agent B.
- The task keeps the same dialog and task IDs throughout the exchange.

- A simulated disconnect interrupts the exchange.
- The adapter restarts and reconnects.
- The backend reloads the saved state.
- A retry with an already processed request ID is rejected instead of running the operation again.

4. Optimizations Proposed by the Team

The main improvement is to make failure and retry part of the normal workflow. Instead of starting a new task after a restart, the system will find the saved task and continue from its last known state.

Area	Without our mechanism	With our approach
Task identity	A retry may look like a new task.	Dialog ID and task ID stay with the task.
Restart	The current state may be lost.	The saved state is loaded again.
Duplicate request	The same operation may run twice.	The request ID is checked first.
Failure	Recovery is mostly manual.	Disconnect and retry cases are tested directly.
State changes	Unexpected transitions can slip through.	Only defined transitions are accepted.

The final version will also show the difference between a normal request and a request that arrives again after a failure.

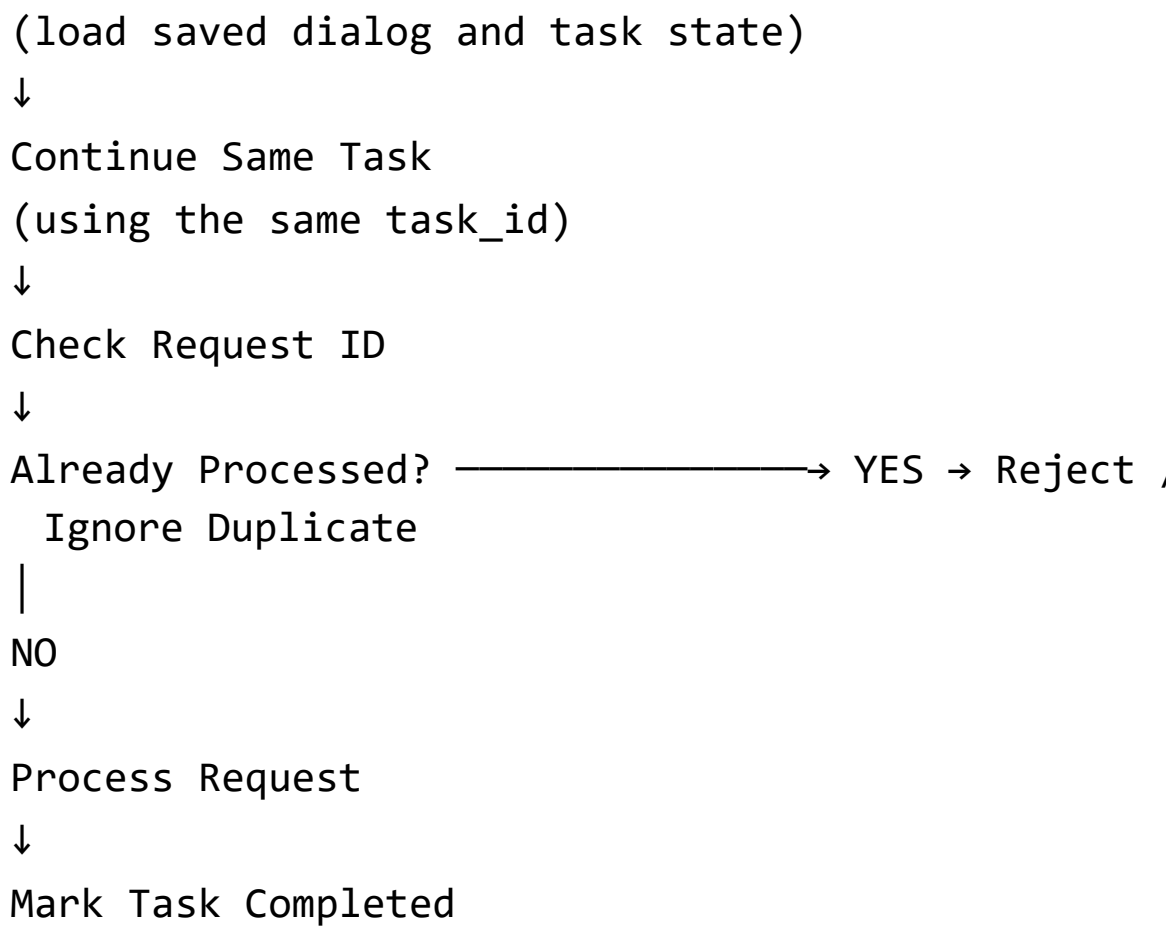
5. Solution Architecture and Design

The first version will run locally so the whole system can be developed and demonstrated without depending on paid cloud infrastructure.

- Client / test script
- Spring Boot dialog manager — IDs, lifecycle, recovery and duplicate checks
- FastAPI Agent Adapter A ↔ FastAPI Agent Adapter B • PostgreSQL — durable dialog, task and request state
- Gemini API — optional LLM layer for agent behaviour
- Docker Compose — runs the services together

Here is the complete flow chart for this project,

```
Client Request
↓
Create IDs
(dialog_id, task_id, request_id)
↓
Route to Agent
(Agent A / Agent B)
↓
Process Task
(Gemini optional)
↓
Persist State
(PostgreSQL / Redis)
↓
Agent Failure
(disconnect / restart)
↓
Restore State
```



Basic flow:

- Create task → assign dialog ID, task ID and request ID.
- Send the request from Agent A to Agent B.
- Save the current state.
- Simulate a disconnect or restart.
- Reconnect and retry the request.
- Reload the saved state and check the request ID.
- Continue the task or reject the duplicate request.

A final architecture diagram will be added after the first end-to-end flow is working.

6. Timeline of Delivery

Stage	Work	Output
1. Setup	GitHub repo, project structure and Docker setup	Working development setup
2. Dialog handling	IDs, states and Spring Boot APIs	Basic task tracking
3. Agent adapters	FastAPI adapters and agent-to-agent exchange	Two connected adapters
4. Persistence	Save and reload task/request state	Restart recovery
5. Deduplication	Request ID checks	Duplicate rejection
6. Failure tests	Five disconnect/retry cases	Test results
7. Submission	PDF, pseudocode and demonstration	Final submission

7. References

[1] S. Kumar and P. R. Cohen, “Towards a fault-tolerant multi-agent system architecture,” in Proceedings of the Fourth International Conference on Autonomous Agents, 2000, pp. 459–466, doi: 10.1145/336595.337570.

[2] J. Ezekiel and A. Lomuscio, “Combining fault injection and model checking to verify fault tolerance, recoverability, and diagnosability in multi-agent systems,” Information and Computation, vol. 254, no. 2, pp. 167–194, 2017, doi: 10.1016/j.ic.2016.10.007.

[3] M. Ganzha, M. Paprzycki, C. Badica, R. Unland, D. Mitrović, Z. Budimac, M. Ivanović, and M. Vidaković, “Agent-based approaches to managing fault-tolerant networks of distributed multi-agent systems,” Multiagent and Grid Systems, vol. 7, no. 6, pp. 203–218, 2011, doi: 10.3233/MGS2011-0176.

[4] H. van Ditmarsch, K. Fruzsa, R. Kuznets, and U. Schmid, “A logic for repair and state recovery in Byzantine fault-tolerant multi-agent systems,” in *Proc. 12th Int. Joint Conf. on Automated Reasoning (IJCAR)*, 2024, pp. 114– 134, doi: 10.1007/978-3-031-63501-4_7.

[5] J.-T. Huang, J. Zhou, T. Jin, X. Zhou, Z. Chen, W. Wang, Y. Yuan, M. R. Lyu, and M. Sap, “On the resilience of LLM-based multi-agent collaboration with faulty agents,” in Proceedings of the 42nd International Conference on Machine Learning, vol. 267, 2025, pp. 26202–26226.

[6] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica, “Why do multi-agent LLM systems fail?,” in Advances in Neural Information Processing Systems, vol. 38, 2025.

8. Conclusion

The prototype will show two agent adapters working on the same task without losing track of it when a connection drops or an adapter restarts. The main focus is practical recovery: keep the task identity, restore the saved state and make sure a retry does not repeat an operation.